

HDOC Day-28

Variable-Length 2D Arrays and Row-Major Traversal in C

Aim

To implement C programs using variable-length 2D integer arrays (VLAs), nested loops, column-wise input, row-major traversal, and conversion of a 2D array into a 1D array. The dimensions are validated to ensure that the number of rows and columns are positive.

Activity 1

Column-Wise Input and Row-Major Output of a 2D Array

Problem Statement

1. Accept the number of rows and columns from the user.
2. Ensure that both dimensions are positive. If an invalid value is entered, ask for it again.
3. Declare a variable-length 2D integer array using the entered dimensions.
4. Read the elements column by column using nested loops, with the outer loop over columns and the inner loop over rows.
5. Display the elements in row-major order using a single loop and a linear index.

Algorithm

6. Start the program.
7. Input the number of rows.
8. If rows is less than or equal to 0, repeatedly ask until a positive value is entered.
9. Input the number of columns.
10. If columns is less than or equal to 0, repeatedly ask until a positive value is entered.
11. Declare a variable-length array `arr[rows][cols]`.
12. Use nested loops for column-wise input: outer loop over columns and inner loop over rows.
13. Use a single loop from 0 to `rows * cols - 1`.
14. Convert the linear index using `row = i / cols` and `column = i % cols`.
15. Print the elements in row-major order.
16. Stop the program.

C Program

```
#include <stdio.h>

int main()
{
    int rows, cols, i, j;

    do
    {
        printf("Enter number of rows: ");
        scanf("%d", &rows);
        if (rows <= 0)
```

```

        printf("Rows must be positive. Try again.\n");
    } while (rows <= 0);

    do
    {
        printf("Enter number of columns: ");
        scanf("%d", &cols);
        if (cols <= 0)
            printf("Columns must be positive. Try again.\n");
    } while (cols <= 0);

    int arr[rows][cols];
    printf("\nEnter array elements column by column:\n");

    for (j = 0; j < cols; j++)
        for (i = 0; i < rows; i++)
        {
            printf("Enter element [%d][%d]: ", i, j);
            scanf("%d", &arr[i][j]);
        }

    printf("\nArray in row-major order:\n");
    for (i = 0; i < rows * cols; i++)
    {
        printf("%d ", arr[i / cols][i % cols]);
        if ((i + 1) % cols == 0)
            printf("\n");
    }
    return 0;
}

```

Test Case Table - Activity 1

Test Case	Rows	Columns	Elements Entered Column-Wise	Expected Row-Major Output
1	2	3	1 2 3 4 5 6	1 3 5 / 2 4 6
2	3	2	10 20 30 40 50 60	10 40 / 20 50 / 30 60
3	1	4	5 10 15 20	5 10 15 20
4	-2, then 2	2	1 2 3 4	Reject -2; then 1 3 / 2 4
5	2	0, then 2	1 2 3 4	Reject 0; then 1 3 / 2 4

Activity 2

Copying a Variable-Length 2D Array into a 1D Array

Problem Statement

Write a C program that reads elements into a variable-length 2D array, copies the elements into a 1D array in row-major order, and displays both arrays.

Algorithm

17. Start the program.
18. Input the number of rows and columns.

19. Ensure that both dimensions are positive.
20. Declare a variable-length 2D array `arr[rows][cols]`.
21. Declare a 1D array of size `rows * cols`.
22. Use nested loops to input the elements of the 2D array.
23. Display the original 2D array.
24. Set a linear index `k = 0`.
25. Traverse the 2D array in row-major order.
26. Copy each element into the 1D array and increment `k`.
27. Display the resulting 1D array.
28. Stop the program.

C Program

```
#include <stdio.h>

int main()
{
    int rows, cols, i, j, k = 0;

    do
    {
        printf("Enter number of rows: ");
        scanf("%d", &rows);
        if (rows <= 0)
            printf("Rows must be positive. Try again.\n");
    } while (rows <= 0);

    do
    {
        printf("Enter number of columns: ");
        scanf("%d", &cols);
        if (cols <= 0)
            printf("Columns must be positive. Try again.\n");
    } while (cols <= 0);

    int arr[rows][cols];
    int oneD[rows * cols];

    printf("\nEnter elements of the 2D array:\n");
    for (i = 0; i < rows; i++)
        for (j = 0; j < cols; j++)
        {
            printf("Enter element [%d][%d]: ", i, j);
            scanf("%d", &arr[i][j]);
        }

    printf("\n2D Array:\n");
    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < cols; j++)
            printf("%d ", arr[i][j]);
        printf("\n");
    }

    for (i = 0; i < rows; i++)
        for (j = 0; j < cols; j++)
            oneD[k++] = arr[i][j];

    printf("\n1D Array in row-major order:\n");
```

```

    for (i = 0; i < rows * cols; i++)
        printf("%d ", oneD[i]);
    printf("\n");
    return 0;
}

```

Test Case Table - Activity 2

Test Case	Dimensions	2D Array	Expected 1D Array
1	2 x 3	1 2 3 / 4 5 6	1 2 3 4 5 6
2	3 x 2	10 20 / 30 40 / 50 60	10 20 30 40 50 60
3	2 x 2	5 10 / 15 20	5 10 15 20
4	1 x 5	2 4 6 8 10	2 4 6 8 10
5	3 x 1	7 / 8 / 9	7 8 9

Result

Both C programs were successfully designed using variable-length arrays and loops. In Activity 1, the program validates the dimensions, accepts the 2D array elements column by column, and displays them in row-major order using a single linear loop. In Activity 2, the program stores elements in a variable-length 2D array and copies them into a 1D array in row-major order, displaying both arrays as required.